Emergency Training Simulator POC

Markdown documentation PDF copy

Read-through script

Source: docs/presentations/project-overview-read-through-script.md

Markdown stays editable. This PDF is the reading/share copy.

Boundary: this project is still a POC, not operational guidance.

Read-through script

This is a full spoken version of the project update. It is written to sound natural, not corporate.

Opening

Hey everyone. This project is a proof-of-concept for an AI-driven adaptive emergency-response training simulator.

I am framing this mainly for Dr. Cameron, so I am treating it like a project progress update: what I built, what it proves, what it does not prove yet, and what the next careful step should be.

The easiest way to explain it is this: I am not trying to build the final simulator in one jump. I am trying to prove the core loop first.

That loop is: a trainee does something, the backend receives it, the scenario changes, the system checks what happened, the client gets the new state, and the session gets logged so someone can review it later.

That sounds simple, but honestly, that is the spine of the whole project.

Why start here

With a project like this, there are a lot of shiny directions I could run toward immediately.

Unreal Engine. AI agents. 3D generated assets. Sensors. Projection mapping. Physics. All of that is exciting, and yes, eventually I do want to fight and wrestle with those monsters.

But if I start there too early, I risk building something that looks impressive while the actual system logic underneath is still unclear.

So I started with the boring-but-important part: can the backend keep track of what happened, update state predictably, and explain it afterward?

That is why this first version is backend-first.

Sprint 0

The first phase was Sprint 0, which was basically setting up the rails.

I added the repo structure, documentation, architecture notes, message schemas, scenario files, validation rule files, backend placeholders, Unreal placeholder folders, and asset pipeline placeholders.

The point was not to overbuild. It was to make the project understandable before adding behavior.

That gave the repo a clean shape: backend, scenarios, validation, schemas, docs, scripts, logs, Unreal contract, and assets.

The working backend slice

After that, I implemented the first backend vertical slice.

Right now, the backend uses FastAPI. It has a health endpoint and a WebSocket session endpoint.

A trainee action can come in over WebSocket. The backend validates the message shape, applies the action through a deterministic scenario engine, runs POC validation rules, sends the updated scenario state back to the client, and writes a JSONL log entry.

There are also replay and summary scripts, so after a session finishes, the log can be inspected and turned into a review summary.

That is the key proof point. It is not just "did the demo run?" It is "can I replay and explain what happened?"

Deterministic engine

The scenario engine is deterministic on purpose.

That means the same ordered inputs should produce the same outputs.

For a training system, that is really useful early on because it makes testing, replay, and debugging much easier.

I am intentionally not using randomness, real AI calls, or external APIs in this slice. Those can come later, once the basic trust layer is solid.

Security baseline

I also added the first security baseline.

The simple version is: the client can ask to attempt an action, but the backend decides what actually happened.

That matters because a future Unreal client should not be able to say, "trust me, the scenario is complete," or "trust me, there were no warnings."

The backend owns the session state, validation result, final outcome, log entries, replay result, and review summary.

Right now, the backend rejects unexpected fields, weird IDs, path-like scenario names, unsafe action names, oversized payloads, wrong sequence numbers, and duplicate event IDs.

This is not production security yet. There is no authentication, authorization, TLS, cloud hardening, or real user data policy.

But it is the first lock on the first door, and it sets the right habit before the project gets bigger.

The current scenario

The current scenario is a tiny smoke-inhalation room POC.

I want to be clear: this is not medical guidance and it is not firefighter or EMT protocol authority. It is a software scenario used to prove the backend loop.

The happy path is simple:

First, assess the scene. Then confirm PPE. Then request backup. Then move the wooden chair that is blocking the path. Then open the entry door. Then enter the room. Then remove the occupant.

Again, it is intentionally small. The purpose is not realism yet. The purpose is clean state transitions.

Environment objects

One important step I added was environment object state.

The scenario now has a wooden chair and an entry door.

The chair can block the path, and the door can be open or closed.

This matters because emergency training scenarios should not disregard environmental cues. Even in this toy version, the room should matter.

If the trainee tries to enter while the chair is still blocking the path or while the door is still closed, the backend can return a warning or error instead of pretending everything is fine.

Physics preview

There is also a physics preview concept for the wooden chair, but it is metadata only right now.

So when the demo says physics enabled or disabled, it does not mean there is a real physics engine running.

It means the object is being marked as a future physics candidate, with notes for things like collision, mass, force direction, tipping behavior, and eventually gravity.

Mass alone is not enough for gravity. Mass tells me how heavy the object is. Gravity explains how that mass behaves in a world with downward acceleration. Later, a real physics engine would need both object properties and world physics settings.

For this slice, I am only capturing the idea in metadata.

Door slice

The door was the next obvious object.

Right now, the main thing that matters is open versus closed.

Door type, lock state, hinge side, swing direction, collision shape, mass, and force direction are metadata for later.

That keeps the scenario simple while still pointing toward richer environment behavior.

Unreal contract

The repo also has a thin Unreal client contract.

The key rule is that Unreal is not the source of truth.

The backend owns scenario logic. Unreal, later, should connect, send trainee actions, receive state updates, and display the current state.

For now, there is no Unreal scene. There is a Python contract harness and a text-only display preview that behaves like the future client.

That lets me test the client contract without getting trapped in engine work too early.

Sample happy-path demo result

In the happy-path demo, the trainee follows the expected sequence.

The system shows that the chair was moved, the door was opened, the final phase reaches occupant removed, there are no warnings or errors, and replay verification passes.

That proves the clean path through the system.

Sample warning/error result

In the warning/error demo, the trainee tries to do something too early or ignores an environmental cue.

The backend flags the issue. The state does not incorrectly advance. The log still records the attempted action.

That is important because a training system should be able to discuss mistakes, not just successful runs.

The review summary can say what actions were taken, what warnings appeared, what errors happened, what the final state was, and whether the end condition was reached.

What comes next

The next goal is to keep strengthening the display contract and environment object contract before jumping into full Unreal work.

I want the future Unreal side to have a very boring job at first: connect, receive state, show clear text, and send simple inputs.

At the same time, I want to keep tightening the security boundary so a future client is useful without becoming too trusted.

Once those contracts are stable, then it makes sense to install Unreal Engine, bring in Blender, experiment with generated assets, and eventually test physics.

Basically: backend truth first, security boundary close behind it, display second, assets third, physics after the object model is stable.

Ideal end result

The ideal end result is an adaptive training simulator where trainees act inside a scenario, the environment responds, cues matter, the instructor can review the session, and AI can eventually help generate or adapt safe training content.

Unreal would handle the visual experience.

The backend would own scenario state and review.

Generated assets would go through manual review before being used.

And every session would leave behind a useful record for learning.

Closing

So that is where the project is right now.

It is not trying to be the final training product yet.

It is proving the foundation: action in, state update, rule check, client message, log, replay, review.

That foundation is small, but it is strong enough to build on.

And from here, the plan is to keep adding one careful layer at a time.

One monster at a time.

If I were presenting this live, I would keep it simple: about five minutes explaining the loop and the scope, then about five minutes running the local demo.

The main thing I would want Dr. Cameron to see is that the foundation works: the backend can receive actions, update state, check the result, log it, replay it, and explain it afterward.